DEPARTMENT OF COMPUTER SCIENCE

SS-CASE-IT

Lab 4

Learning Objectives

1. Breadth First Search on Graph.
2. Depth First Search on Graph.

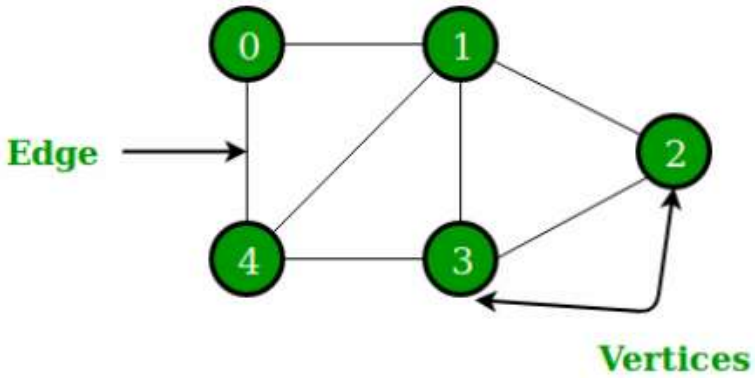
---

1. Graph

A graph is collection of two sets V and E where V is a finite non-empty set of vertices and E is a finite non-empty set of edges.

- Vertices are nothing but the nodes in the graph.
- Two adjacent vertices are joined by edges.
- Any graph is denoted as G = {V, E}.

For example:

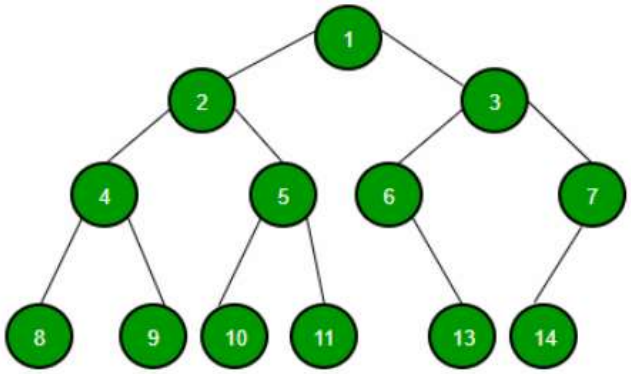


G = {{V1, V2, V3, V4, V5, V6}, {E1, E2, E3, E4, E5, E6, E7}}

Whereas, a tree is a finite set of one or more nodes such that:

- There is a specially designated node called root.
- The remaining nodes are partitioned into n>=0 disjoint sets T1, T2, T3, …, Tn
  where T1, T2, T3, …, Tn is called the subtrees of the root.

The concept of tree is represented by following Fig.



There are different types of blind or uninformed searches that can be used to solve a certain problem.
Commonly used search techniques are:

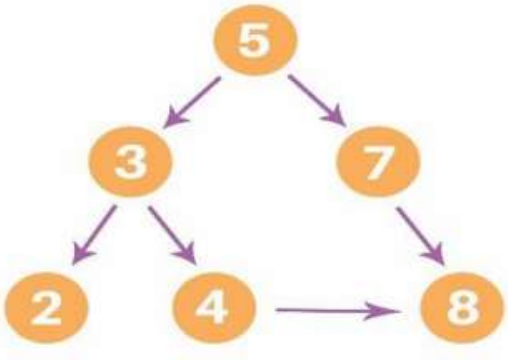
1. Breadth First Search
2. Depth First Search

2. Breadth First Search

The BFS search starts from a node, then it checks all the nodes at distance one from the beginning node, then it checks all the nodes at distance two, and so on. So as to recollect the nodes to be visited, BFS uses a queue.

The steps of the algorithm work as follow:

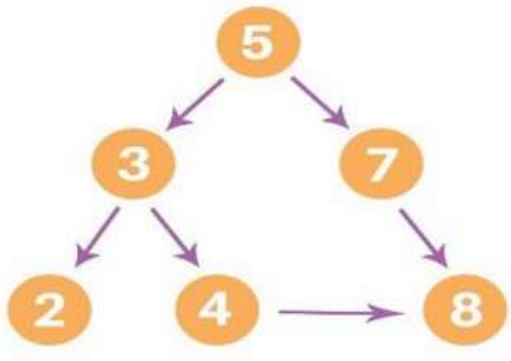
1. Start by putting any one of the graph's vertices at the back of the queue.
2. Now take the front item of the queue and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add those which are not within the visited list to the rear of the queue.
4. Keep continuing steps two and three till the queue is empty.



3. Depth First Search

The Depth-First Search is a recursive algorithm that uses the concept of backtracking. It involves thorough searches of all the nodes by going ahead if potential, else by backtracking. Here, the word backtrack means once you are moving forward and there are not any more nodes along the present path, you progress backward on an equivalent path to seek out nodes to traverse. All the nodes

are progressing to be visited on the current path until all the unvisited nodes are traversed after which subsequent paths are going to be selected.
Before learning the python code for Depth-First and its output, let us go through the algorithm it follows for the same. The recursive method of the Depth-First Search algorithm is implemented using stack.



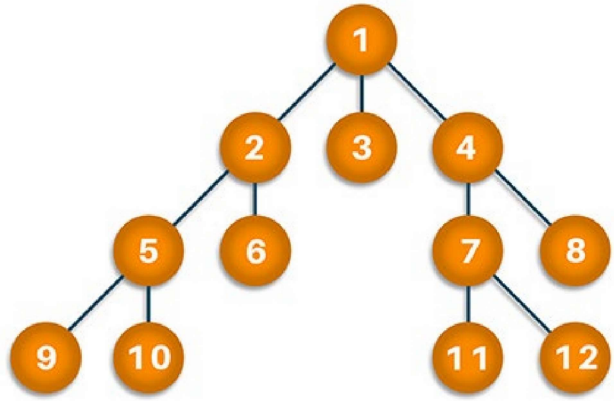
The **DSF algorithm** follows as:

1. We will start by putting any one of the graph's vertex on top of the stack.
2. After that take the top item of the stack and add it to the visited list of the vertex.
3. Next, create a list of that adjacent node of the vertex. Add the ones which aren't in the visited list of vertexes to the top of the stack.
4. Lastly, keep repeating steps 2 and 3 until the stack is empty.

Note: Solution of Task 1 is going to be demonstrated in Lab and Task 2 + 3 is your Lab Activity.

Task 1:

Write a python program to implement DFS and BFS on the graph given below. Consider this to be a directed graph. Start state =1 and Goal state = 11. Your program should stop once Goal state is reached.

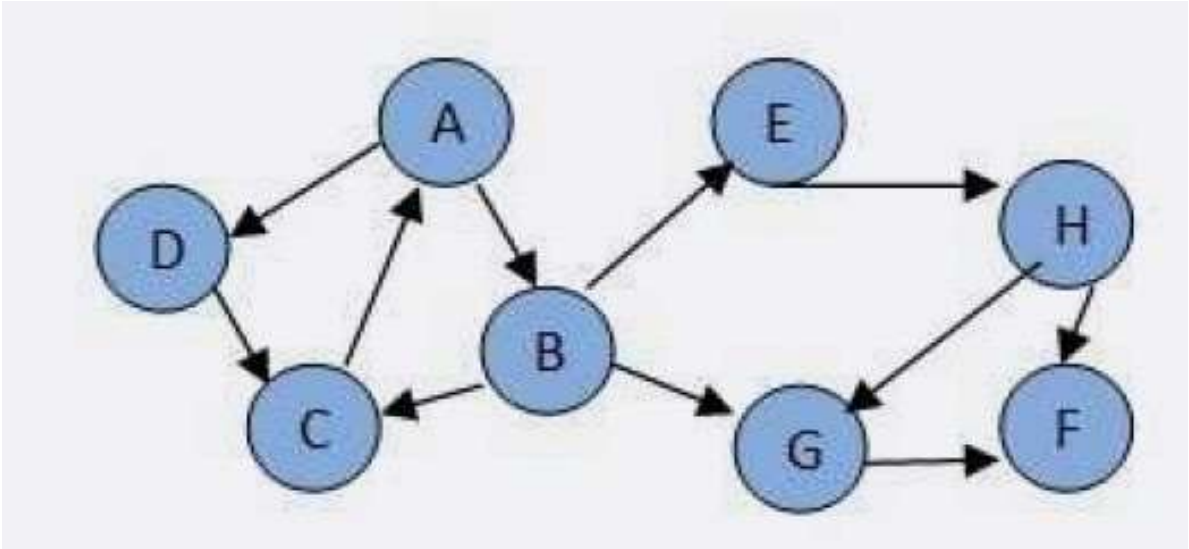


Task 2:

Write a python program to implement DFS and BFS on the graph given below. Input should be taken from the user.

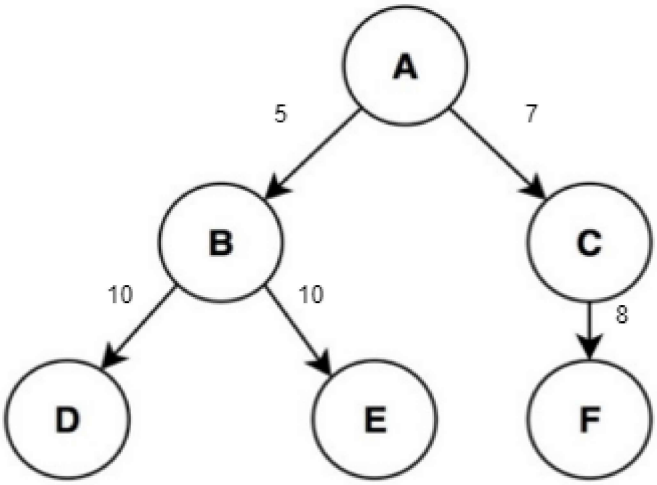
Start state: A, Goal state: G.

When the program reaches the Goal state, the traversal should stop and prints the path.



Task 03:

This is your home task. Modify the above code of BFS and DFS in such a way so that cost associated with each path be calculated. The goal node should return the order of expansion and the total path cost. An example question is given below.



Viva:

```
sampleDict={
    "class":{
        "student":{
            "name":"Mike",
            "marks":{
                "physics":70,
                "history":80

            }
        }
    }
}
  # Write print() to get the following output from the above defined Dictionary?
```

```
{'name': 'Mike', 'marks': {'physics': 70, 'history': 80}}
```